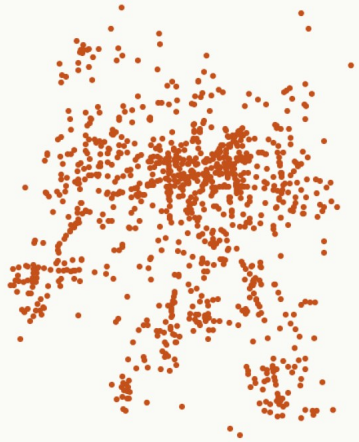
GeoPandas y redes
Ayudantía 8

GeoPandas

Datos geoespaciales

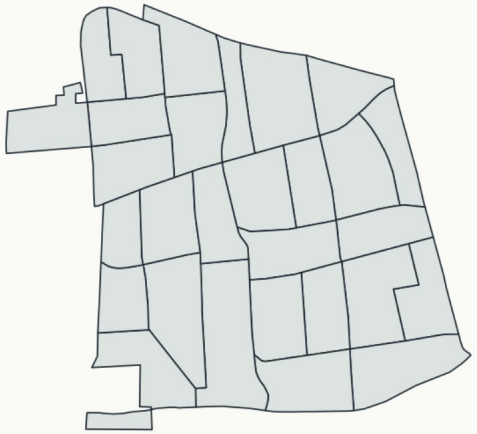
Una tabla común, como una planilla de Excel, tiene filas y columnas. Un dato geoespacial es lo mismo con una columna extra, la que guarda la forma de cada fila sobre el mapa. Gracias a esa columna, cada fila sabe dónde está.



Puntos: accidentes de bicicleta



Líneas: tramos del metro



Polígonos: distritos de Santiago

Cada capa usa un solo tipo de forma. Para cruzar accidentes (puntos) con distritos (polígonos) trabajamos con dos capas distintas y las combinamos después.

Dataset de ejemplo

Capa	Qué representa	Cada fila es
Comunas	Límites de las 52 comunas de la región	una comuna
Distritos censales	449 distritos con datos socioeconómicos	un distrito
Áreas urbanas	Zonas urbanas del país	una localidad
Estaciones de metro	127 estaciones	una estación
Líneas de metro	137 tramos entre estaciones	un tramo
Accidentes	Atropellos, bicicletas, motos y puntos críticos	un lugar
Seguridad	121.351 puntos con un puntaje de percepción	un punto
Delitos	11 tipos de delito, mes a mes	una comuna

Accidentes

```
b = gpd.read_file(ruta_bicicletas)
print(len(b), "filas")
print(b["Accidentes"].sum(), "accidentes")
```

```
987 filas
1118 accidentes
```

```
b[["Calle1", "Accidentes",
"Graves"]].head(3)
```

```
            Calle1  Accidentes  Graves
0  EL CONQUISTADOR           1       0
1       LA GALAXIA           1       0
2       LA GALAXIA           1       1
```

Archivo	Año	Filas	Accidentes
Atropellos	2020	981	1.111
Bicicletas	2019	987	1.118
Motocicletas	2019	2.168	2.651
Puntos críticos	2019	802	6.928

Cada fila es un lugar, un cruce o un tramo de calle, y no un accidente. La columna Accidentes dice cuántos hubo ahí. En el archivo de atropellos esa columna se llama Atropellos.

También hay columnas por gravedad: fallecidos, graves, menos graves y leves. Un solo punto crítico llega a 57 accidentes, por eso contar filas y sumar accidentes da resultados distintos.

Distritos y grupos socioeconómicos

```
cols = ["NOM_DIS", "E", "D", "C3", "C2", "ABC1", "ADIMARK_T"]
distritos[cols].head(3)
```

	NOM_DIS	E	D	C3	C2	ABC1	ADIMARK_T
0	Huelén	28.0	397.0	1232.0	1390.0	365.0	3412.0
1	Moneda	51.0	560.0	999.0	903.0	228.0	2741.0
2	Amunátegui	38.0	389.0	956.0	891.0	113.0	2387.0

```
suma = distritos[["E", "D", "C3", "C2",
"ABC1"]].sum(axis=1)
print((suma ==
distritos["ADIMARK_T"]).all())
```

```
True
```

Cada fila es un distrito. Las columnas E, D, C3, C2 y ABC1 son grupos socioeconómicos. Según la clasificación habitual en Chile, E es el de menor ingreso y ABC1 el de mayor.

ADIMARK_T es la suma de los cinco grupos, como comprueba la celda. En toda la región suma 1,65 millones, bastante menos que su población, así que no cuenta personas. Su unidad no está documentada.

Percepción de seguridad

```
seguridad = pd.read_csv(ruta_seguridad)
print(len(seguridad), "puntos")
seguridad.head(3)
```

```
121351 puntos
   seguridad        lat        lon
0  -0.120456 -33.323944 -70.512634
1  -0.103042 -33.323944 -70.512729
2   0.391172 -33.323944 -70.512987
```

Cada fila es un punto de la ciudad, con su latitud, su longitud y un puntaje de percepción de seguridad. Todavía no es una capa de mapa: hay que crear los puntos a partir de lat y lon.

```
seguridad["seguridad"].agg(["mean", "std",
"min", "max"]).round(2)
```

```
mean    0.00
std     1.00
min    -4.41
max     4.21
Name: seguridad, dtype: float64
```

El puntaje está estandarizado: promedio 0 y desviación 1. Los archivos no indican si un valor alto significa más seguridad. El análisis considera que un valor más bajo es una peor percepción.

Delitos

```
delitos = gpd.read_file(ruta_delitos)
delitos[["UnddTrr", "enr2018",
"Total"]].head(3)
```

	UnddTrr	enr2018	Total
0	SANTIAGO	102.593697	3254.264992
1	CERRILLOS	45.112260	2873.248051
2	CERRO NAVIA	39.186349	1882.717475

Cada fila es una comuna. Hay una columna por mes, de enr2018 a dcm2020, con 36 columnas, y Total es exactamente su suma.

Los valores son decimales, así que no son conteos de casos, y el archivo no explica su unidad. Como están por comuna, no se pueden ubicar en una calle ni en un nodo de la red.

Hay 11 tipos de delito, uno por archivo. En los cuadernos se usan dos: robo con violencia o intimidación y hurtos.

Cargar un archivo

```
import geopandas as gpd

# lee el archivo de mapa
distritos = gpd.read_file(ruta)
# nos quedamos con 3 columnas
cols = ["NOM_COM", "NOM_DIS",
"ADIMARK_T"]
distritos[cols].head(3)
```

	NOM_COM	NOM_DIS	ADIMARK_T
0	Santiago	Huelén	3412.0
1	Santiago	Moneda	2741.0
2	Santiago	Amunátegui	2387.0

```
# solo los distritos de Santiago
santiago = distritos[
    distritos["NOM_COM"] == "Santiago"]
santiago.plot()
```


Los mapas suelen venir como shapefile, que en realidad son varios archivos con el mismo nombre (.shp, .dbf, .shx y .prj). Basta indicar el .shp y GeoPandas encuentra los demás.

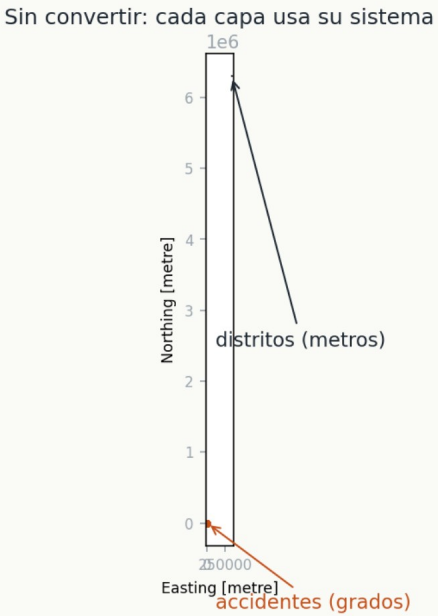
Lo que obtienes es un GeoDataFrame, una tabla de pandas con una columna geometry. Todo lo que sabes de pandas, como filtrar con corchetes u ordenar, sigue funcionando. Un CSV, en cambio, no guarda formas: si trae latitud y longitud hay que convertirlas en puntos.

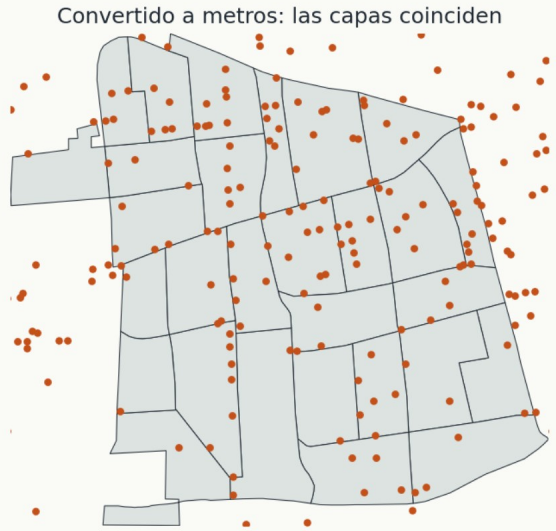
Sistemas de coordenadas

El mismo lugar, la Plaza de Armas, se puede escribir de dos maneras: en grados, (-33,4378; -70,6504), o en metros, (346.584; 6.298.959).

Cada manera es un sistema de coordenadas, o CRS: EPSG:4326 usa grados y EPSG:32719 usa metros. Si mezclas capas con CRS distintos, quedan separadas, como a la izquierda del gráfico.

Antes de combinar, convierte todo al mismo CRS.





Revisar y cambiar el CRS

```
# ¿en qué sistema está cada capa?
print(distritos.crs)
print(accidentes.crs)
```

```
EPSG:32719
EPSG:4326
```

```
# límites de la capa, en sus unidades
distritos.total_bounds.round().astype(int)
```

```
array([ 249135, 6205240,  428357, 6355827])
```

```
# lo mismo con los accidentes
accidentes.total_bounds.round(2)
```

```
array([-70.8 , -33.63, -70.51, -33.33])
```

```
# pasamos los accidentes a metros
accidentes = accidentes.to_crs(32719)
accidentes.total_bounds.round().astype(int)
```

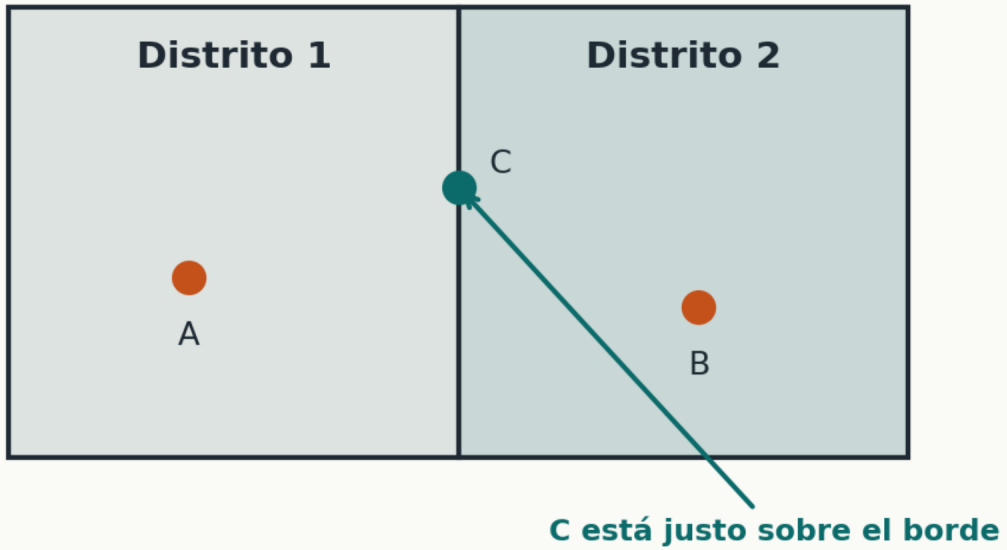
```
array([ 333116, 6278053,  359253, 6310956])
```

Los distritos están en metros: números de millones. Los accidentes llegaron en grados: números de decenas. Después de to_crs, ambas capas hablan en metros y se pueden combinar.

Spatial join

El spatial join une dos tablas usando la ubicación en lugar de un nombre. Para cada punto busca el polígono donde cae y le copia sus columnas, como el nombre del distrito.

Hay que decidir qué significa «caer en». A y B están dentro de un distrito, pero C está sobre el borde: con within se descarta, y con intersects cuenta para los dos distritos.



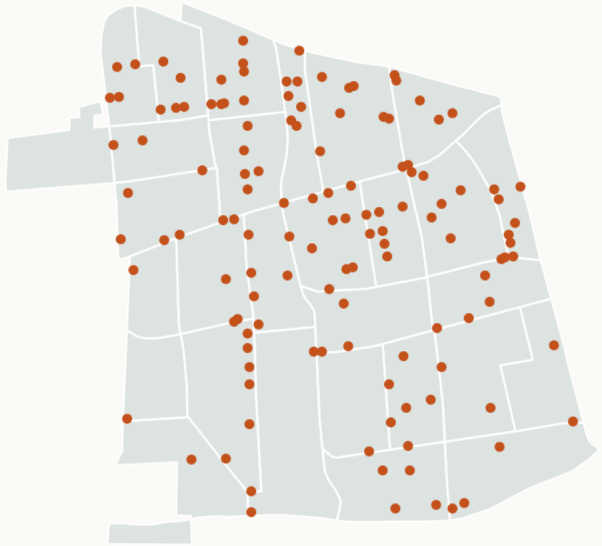
Hacer el spatial join

```
# mismas unidades que los distritos
puntos = accidentes.to_crs(32719)

# ¿en qué distrito cae cada punto?
j = gpd.sjoin(
    puntos, santiago,
    predicate="within")

j[["Calle1", "Calle2", "NOM_DIS"]].head(4)
```

	Calle1	Calle2	NOM_DIS
297	MATUCANA	SAN PABLO	Matucana
299	MATUCANA	CATEDRAL	Portales
303	MAPOCHO	CHACABUCO	Matucana
304	MATUCANA	NULL	Chacabuco



De los 987 accidentes de la región, solo 130 caen dentro de los distritos de Santiago. Los demás ocurrieron en otras comunas y no encuentran polígono, así que se descartan.

Merge

```
# una clave igual en las dos tablas
comunas["clave"] = (
    comunas["Comuna"].apply(norm))
delitos["clave"] = (
    delitos["UnddTrr"].apply(norm))

# unimos por la clave
union = comunas.merge(
    delitos[["clave", "Total"]],
    on="clave", how="left")
```

```
print(len(union), "filas")
print(union["Total"].isna().sum(), "sin
dato")
```

```
52 filas
0 sin dato
```

```
union[["Comuna", "Total"]].head(3)
```

	Comuna	Total
0	San Joaquín	2187.861663
1	San Miguel	2940.518662
2	San Ramón	2194.446731

merge une dos tablas buscando filas con el mismo valor en una columna, la clave. Es parecido a BUSCARV en Excel.

how="left" conserva todas las filas de la tabla de la izquierda, las 52 comunas. Si una no tuviera pareja quedaría con NaN, por eso se revisa. Con how="inner" esas filas desaparecerían.

Overlay

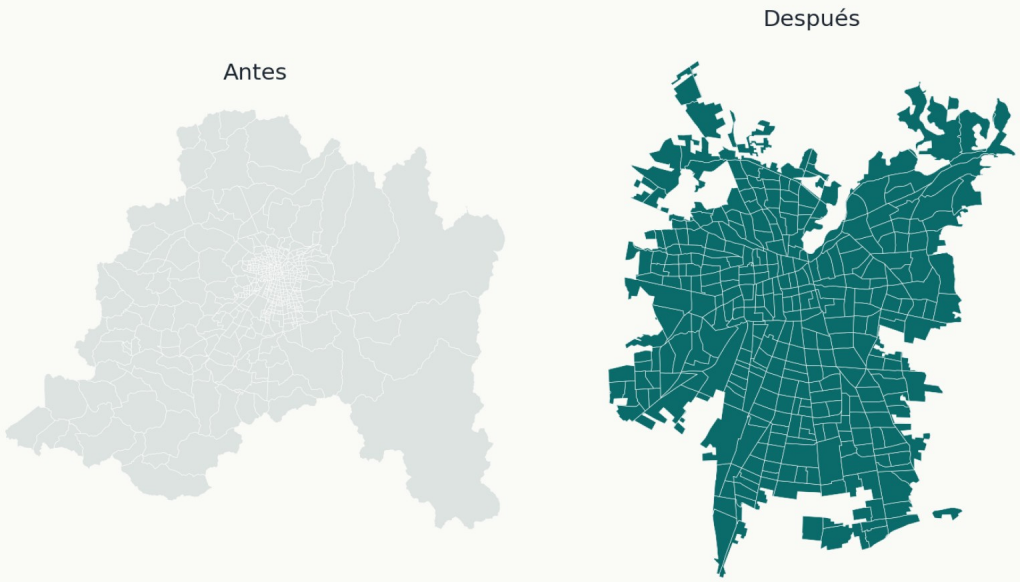
```
# el área urbana de Santiago
urbana = areas[
    areas["NOMBRE"] == "Santiago"]

# recortamos los distritos con ella
cortado = gpd.overlay(
    distritos, urbana,
    how="intersection")

len(distritos), len(cortado)
```

```
(449, 340)
```

A diferencia del spatial join, que solo copia columnas, overlay crea formas nuevas. Por eso las áreas y densidades originales ya no describen las piezas y hay que recalcularlas.



Una ciudad no ocupa todo el territorio de sus distritos, porque parte de ellos son cerros o zonas rurales. Overlay recorta una capa con otra, como unas tijeras: de los 449 distritos de la región conserva solo lo que cae dentro del área urbana, y quedan 340 piezas.

Leer los resultados

Antes de concluir, mira qué se perdió: de 987 accidentes solo 130 cayeron en Santiago.

Luego compara con justicia: Almagro lidera en conteo y Club Hípico en tasa. Parque O'Higgins empata con Almagro con solo 2 accidentes, porque su distrito es pequeño.

Y di qué límites tiene: un solo año, solo bicicletas, y ADIMARK_T sin confirmar.

Distrito	Accidentes	ADIMARK_T	Tasa
Club Hípico	7	745	94
Victoria	6	1.434	42
Almagro	10	2.942	34
Parque O'Higgins	2	594	34

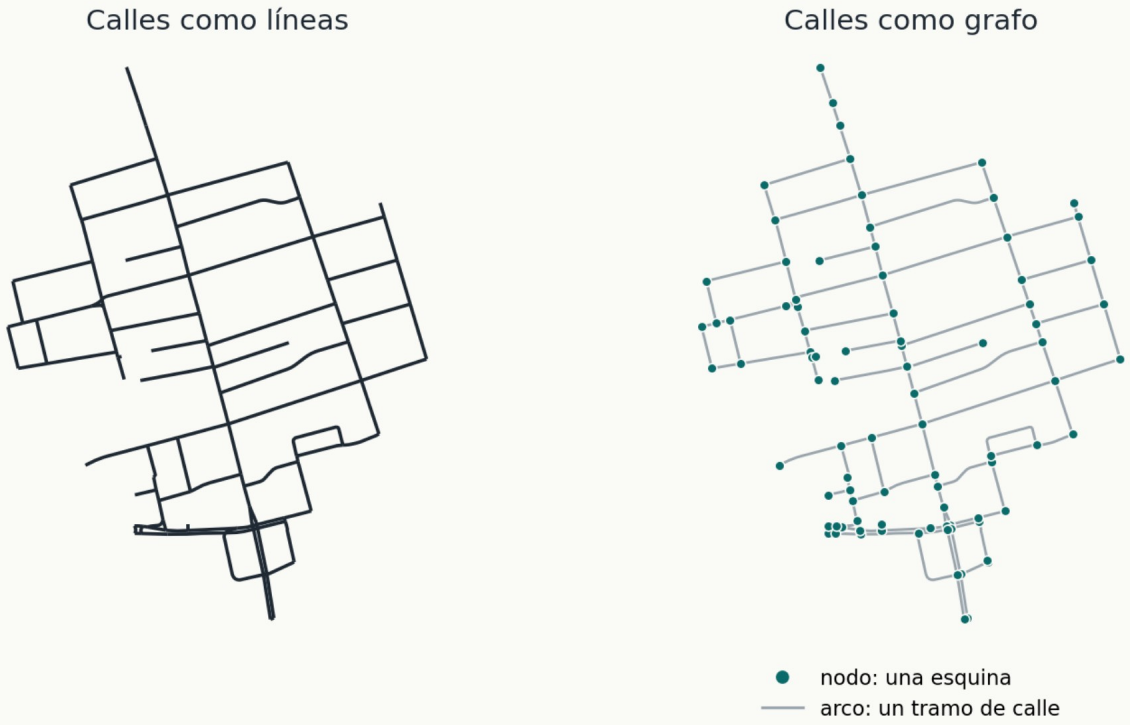
Redes

Calles y grafo

En GeoPandas una calle es una línea con su forma, pero esa línea no sabe con qué otras calles se conecta. Para saber hacia dónde se puede seguir en cada esquina, las calles se convierten en un grafo.

Los dos dibujos muestran las mismas calles de Providencia. A la izquierda son líneas. A la derecha, cada esquina es un nodo y cada tramo entre esquinas es un arco.

Con el grafo se pueden calcular caminos, distancias y qué esquinas son más importantes.

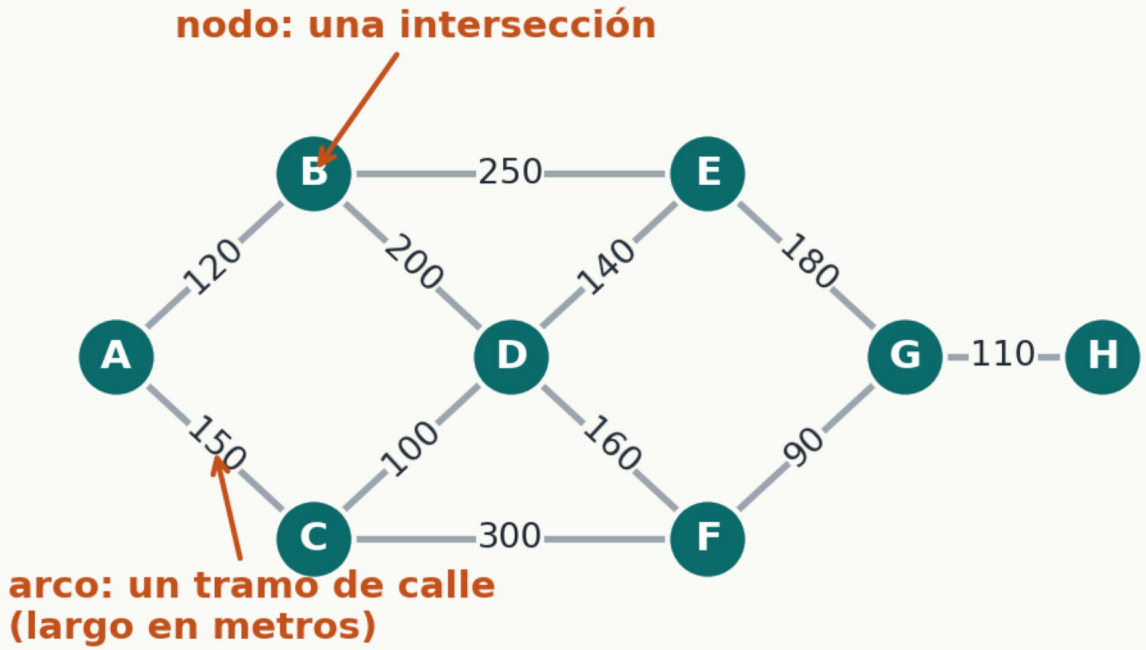


Qué es un grafo

Piensa en el mapa del metro. No respeta las distancias reales ni la forma de las calles, pero sirve para lo importante: qué estaciones se conectan con cuáles.

Un grafo hace lo mismo con una ciudad. De todo el mapa guarda solo las intersecciones, que llamamos nodos, y los tramos de calle que las unen, llamados arcos. Cada arco puede traer un número, por ejemplo su largo en metros.

Con esa versión simplificada, preguntas como cuál es el camino más corto de A a H se convierten en cálculos con números, que el computador resuelve rápido.



De dónde vienen las calles



OpenStreetMap es un mapa del mundo que construyen voluntarios y que se puede usar gratis. Guarda cada calle con datos como su nombre, su tipo y su sentido.

OSMnx es la librería de Python que le pide a OpenStreetMap las calles dentro de un polígono, por ejemplo el de una comuna. La consulta viaja por internet a un servidor público llamado Overpass, que responde con las calles, y OSMnx las convierte en un grafo.

Eso es lo que significa descargar la red. Como el servidor es gratuito y lo usa mucha gente, a veces está saturado y la descarga falla o demora. Por eso, después de la primera vez, la red se guarda en un archivo y las siguientes ejecuciones lo leen sin conectarse.

La red descargada

```
G = ox.graph_from_polygon(poligono,
network_type="drive")
print(len(G.nodes), "nodos")
print(len(G.edges), "arcos")
```

```
1617 nodos
3112 arcos
```

Cada nodo es una intersección y cada arco es un tramo de calle. Los arcos traen su largo en metros (length), el tipo de vía (highway), si es de un solo sentido (oneway) y, a veces, el nombre y la velocidad máxima.

El tipo de red cambia qué calles entran. En Providencia, drive tiene 1.617 nodos, walk tiene 6.703 porque incluye veredas y pasajes peatonales, y bike tiene 2.822.

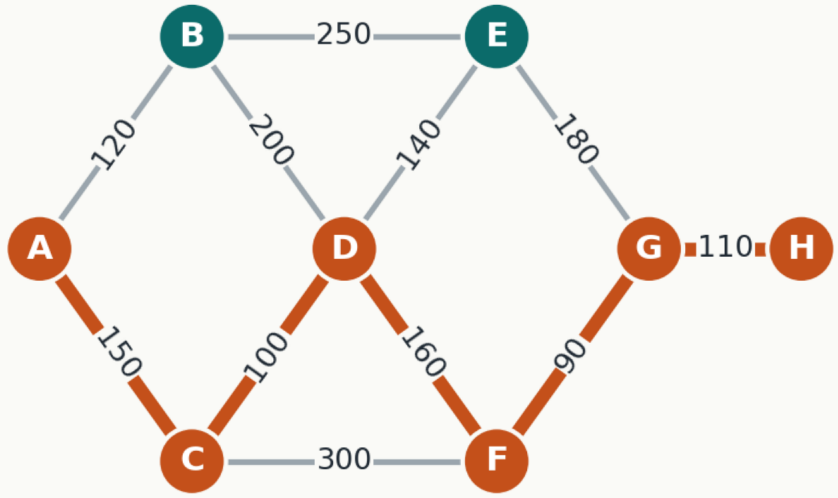
```
nodos, arcos = ox.graph_to_gdfs(G)
arcos[["name", "highway", "length", "oneway"]].reset_index(drop=True).head(3)
```

	name	highway	length	oneway
0	Rengo	residential	57.667718	True
1	Rengo	residential	52.557785	True
2	Avenida José Miguel Claro	tertiary	72.436665	True

Camino más corto

Buscar el camino más corto de A a H significa comparar los recorridos posibles y quedarse con el de menor largo total. En este barrio hay 12 caminos distintos. Los cuatro más cortos son los de la tabla.

Camino	Largo (m)
A, C, D, F, G, H	610
A, C, F, G, H	650
A, B, E, G, H	660
A, B, D, F, G, H	680



El ganador pasa por más nodos que el segundo y aun así es 40 metros más corto, porque evita el tramo de C a F, que mide 300.

Calcular el camino

```
import networkx as nx
G = nx.Graph()
# un tramo: nodos A y B, largo 120 m
G.add_edge("A", "B", length=120)
# ...y así hasta completar 11 tramos
```

```
# camino más corto de A a H
nx.shortest_path(
    G, "A", "H", weight="length")
```

```
['A', 'C', 'D', 'F', 'G', 'H']
```

```
nx.shortest_path_length(
    G, "A", "H", weight="length")
```

```
610
```

Primero se construye el grafo agregando un arco por cada tramo, con su largo guardado en length.

weight="length" le dice a NetworkX qué número de cada arco debe sumar. Sin esa indicación contaría cuántos tramos tiene el camino y no cuántos metros.

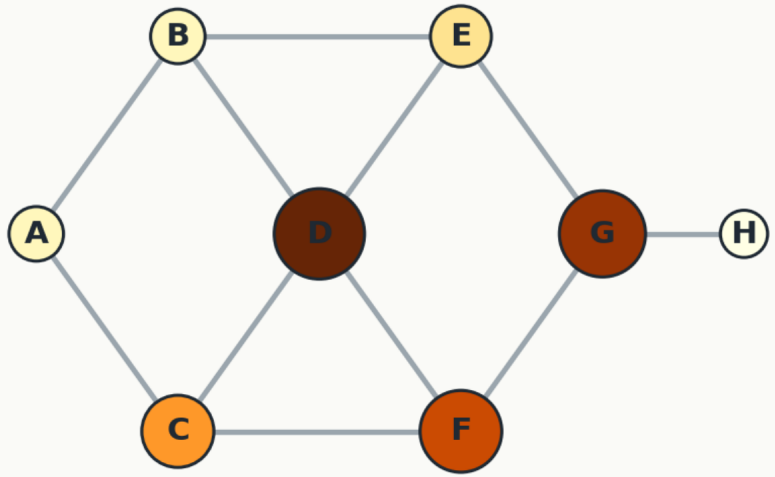
shortest_path devuelve la lista de nodos del camino, y shortest_path_length devuelve su largo total.

Centralidad

```
bc = nx.betweenness_centrality(
    G, weight="length")
cc = nx.closeness_centrality(
    G, distance="length")
# nodo, grado, cercanía, intermediación
for n in ["D", "H"]:
    print(n, G.degree(n),
          round(cc[n], 4), round(bc[n], 2))
```

```
D 4 0.0048 0.38
H 1 0.0027 0.0
```

D gana en las tres medidas. H está en un extremo del barrio: solo le llega una calle, queda lejos de todos y ningún camino mínimo pasa por él.

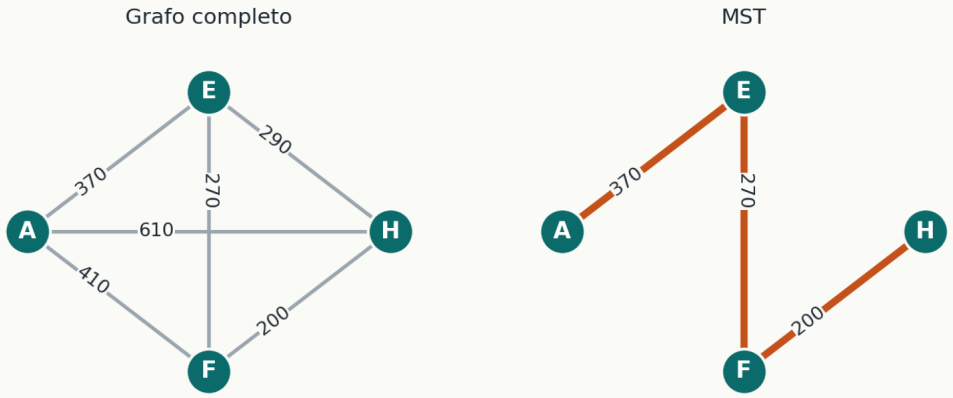


El grado cuenta las calles que llegan al nodo. La cercanía es el inverso de la distancia media a los demás: 0,0048 equivale a estar, en promedio, a unos 209 metros del resto. La intermediación es la fracción de caminos mínimos que pasan por el nodo.

Árbol de expansión mínima

Para unir cuatro lugares con la menor cantidad de calle, primero se mide la distancia real entre cada par. Luego se revisan las conexiones de la más corta a la más larga, y cada una se elige solo si une algo que todavía no estaba conectado.

Conexión	Metros	Decisión
F a H	200	se elige
E a F	270	se elige
E a H	290	se descarta: ya están unidos
A a E	370	se elige
A a F	410	se descarta: ya están unidos
A a H	610	se descarta: ya están unidos



El resultado es un árbol: conecta los cuatro lugares con 840 metros en total y sin ciclos. Es lo más barato, pero no tiene caminos alternativos: si una conexión falla, un lugar queda aislado.

Calcular el árbol

```
puntos = ["A", "E", "F", "H"]
# grafo completo entre los 4 puntos
K = nx.Graph()
for a, b in combinations(puntos, 2):
    d = nx.shortest_path_length(
        G, a, b, weight="length")
    K.add_edge(a, b, weight=d)
# el árbol de expansión mínima
T = nx.minimum_spanning_tree(K)
print(sorted(T.edges()))
print(T.size(weight="weight"))
```

```
[('A', 'E'), ('E', 'F'), ('F', 'H')]
840.0
```

combinations(puntos, 2) genera las 6 parejas posibles de lugares. Para cada pareja se calcula la distancia real por el grafo original y se guarda como peso de un arco.

K es el grafo completo y T es el árbol que queda. El resultado lista las tres conexiones elegidas, las mismas de la tabla anterior, y sus metros sumados.

Para dibujarlo sobre la red original, se recupera el camino de cada conexión con shortest_path.

Flujo máximo

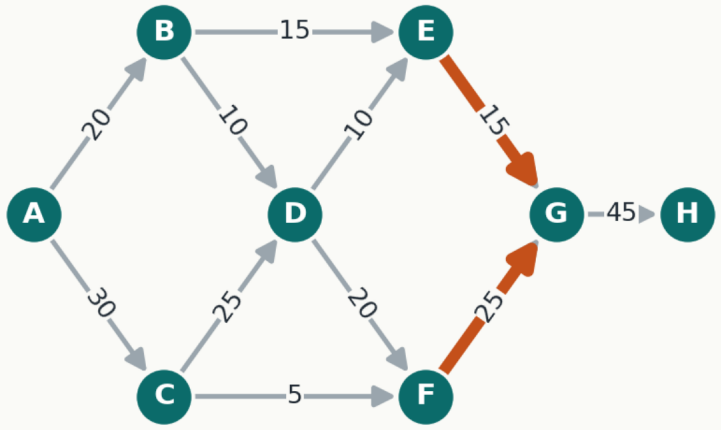
```
D = nx.DiGraph()
# un arco: de A a B, 20 autos por hora
D.add_edge("A", "B", capacity=20)
# ...y así hasta completar 11 arcos
```

```
valor, reparto = nx.maximum_flow(
    D, "A", "H")
valor
```

```
40
```

```
corte, (S, T) = nx.minimum_cut(
    D, "A", "H")
sorted(T)
```

```
['G', 'H']
```



Cada arco tiene una capacidad, por ejemplo cuántos autos por hora pueden pasar. El flujo máximo es lo más que puede ir de A a H sin superar la capacidad de ningún arco. Aquí son 40.

El corte mínimo señala el cuello de botella: los dos arcos que llegan a G, con 15 y 25 de capacidad. Aunque el resto de la red tenga espacio, por ahí no pasan más de 40.